EE 242 Lab 3a – Frequency Domain Representation of Signals - Fourier Series

Team AB41: Aya Burpee, Bryan Schmid, Tyler Watkins, Arturo Games

This lab has 2 exercises to be completed as a team. Each should be given a separate code cell in your Notebook, followed by a markdown cell with report discussion. Your notebook should start with a markdown title and overview cell, which should be followed by an import cell that has the import statements for all assignments. For this assignment, you will need to import: numpy, the wavfile package from scipy.io, and matplotlib.pyplot.

```
# We'll refer to this as the "import cell." Every module you import
should be imported here.
%matplotlib inline
import numpy as np
import matplotlib
import scipy.signal as sig
import matplotlib.pyplot as plt
import cmath
# import whatever other modules you use in this lab -- there are more
that you need than we've included
from scipy.io import wavfile
from scipy import signal
from IPython.display import Audio, display
```

Summary

In this lab, we will learn how to build periodic signals from component sinusoids and how to transform signals from the time domain to the frequency domain. The concepts we'll focus on include: implementation of the Fourier Series synthesis equation, using a discrete implementation of the Fourier Transform (DFT) with a digitized signal, and understanding the relationship between the discrete DFT index k and frequency ω for both the original continuous signal x(t). This is a two-week lab. You should plan on completing the first 2 assignments in the first week.

Lab 3a turn in checklist

• Lab 3a Jupyter notebook with code for the 2 exercises assignment in separate cells. Each assignment cell should contain markdown cells (same as lab overview cells) for the responses to lab report questions. Include your lab members' names at the top of the notebook.

Please submit the report as PDF

Assignment 1 -- Generating simple periodic signals

In the first assignment, you will develop an understanding of how some periodic signals are easier to approximate than others with a truncated Fourier Series. In this lab, we'll work with real signals and use the synthesis equation:

$$x(t) = a_0 + \sum_{k=1}^{N} 2|a_k| cos(k\omega_0 t + \angle a_k)$$

In lecture, you saw that you get ripples at transition points in approximating a square wave (**Gibbs phenomenon**). This happens for any signals with sharp edges. This assignment will involve approximating two signals (a sawtooth and a triangle wave) that have the same fundamental frequency (20Hz).

A. Write a function for generating a real-valued periodic time signal given the Fourier series coefficients $[a_0 a_1 \cdots a_N]$, the sampling frequency, and the fundamental frequency. You may choose to have complex input coefficients or have separate magnitude and phase vectors for describing a_k.

B. Define variables for the sampling frequency (8kHz) and the fundamental frequency (20Hz). Using this sampling frequency, create a time vector for a length of 200ms.

C. The sawtooth signal has coefficients as follows:

$$a_0 = 0.5, a_k = 1/(j2k\pi)$$

Using the function from part A, create three approximations of this signal with N = 2,5,20 and plot together in a 3×1 comparison.

D. A triangle signal has coefficients:

$$a_0 = 0.5, a_k = \frac{2\,sin(k\pi/2)}{j(k\pi)^2} e^{-j2k\pi/2}$$

Create three approximations of this signal with N = 2,5,20 and plot together in a 3×1 comparison.

```
# Assignment 1 - Generating Periodic Signals with Fourier Series

# Part A - Writing a periodic signal generator function
# Input: t = time in seconds, fs = sampling rate, a = COMPLEX
coefficients above, w = fundamental frequency
def fourier_series(t, fs, a, w):
    samples = int(t*fs)
    # avg value array with number of samples as t*f
    x_t = np.ones(samples) * a[0].real
    # time vector
    t = np.linspace(0, t, samples)
    # build fourier series
    for i in range(1, len(a)):
```

```
        x_t += (2*abs(a[i])*np.cos(i*w*t + cmath.phase(a[i])))
    # return x(t)
    return x_t

# Part B - Initialize the parameters
fs = 8000
w0 = 2*np.pi*20
t = 0.2
t_vector = np.arange(0, 0.2, 1/fs)

# Part C - Sawtooth Curve
# Create a vector of a values as shown above for N = 2, 5, 20
# Use the function above to find the approximations
N1 = 2
a1 = np.zeros(N1, dtype = complex)
a1[0] = 0.5
for i in range(1, N1):
    a1[i] = 1/(1j * 2 * np.pi * i)
x1 = fourier_series(t, fs, a1, w0)

N2 = 5
a2 = np.zeros(N2, dtype = complex)
a2[0] = 0.5
for i in range(1, N2):
    a2[i] = 1/(1j * 2 * np.pi * i)
x2 = fourier_series(t, fs, a2, w0)


N3 = 50
a3 = np.zeros(N3,dtype = complex)
a3[0] = 0.5
for i in range(1, N3):
    a3[i] = 1/(1j * 2 * np.pi * i)
x3 = fourier_series(t, fs, a3, w0)

# Plot the 3 approximations
fig, (ax1, ax2, ax3) = plt.subplots(3,1, sharex=True)
fig.suptitle("Sawtooth")
fig.supxlabel("Time (s)")
fig.subplots_adjust(hspace=0.5)

ax1.plot(t_vector, x1)
ax1.set_title("N=2")
ax1.set_ylabel("Amplitude")

ax2.plot(t_vector, x2)
ax2.set_title("N=5")
ax2.set_ylabel("Amplitude")

ax3.plot(t_vector, x3)
```

```
ax3.set_title("N=20")
ax3.set_ylabel("Amplitude")

# Part D - Triangle Curve
# Create a vector of a values as shown above for N = 2, 5, 20
# Use the function above to find the approximations
a1 = np.zeros(N1, dtype = complex)
a1[0] = 0.5
for i in range(1, N1):
    a1[i] = (2*np.sin(i*(np.pi/2)))/ (1j*(i * np.pi)**2)
x1 = fourier_series(t, fs, a1, w0)


a2 = np.zeros(N2, dtype = complex )
a2[0] = 0.5
for i in range(1, N2):
    a2[i] = (2*np.sin(i*(np.pi/2)))/ (1j*(i * np.pi)**2)
x2 = fourier_series(0.2, fs, a2, w0)


a3 = np.zeros(N3,dtype = complex )
a3[0] = 0.5
for i in range(1, N3):
    a3[i] = (2*np.sin(i*(np.pi/2)))/ (1j*(i * np.pi)**2)
x3 = fourier_series(t, fs, a3, w0)

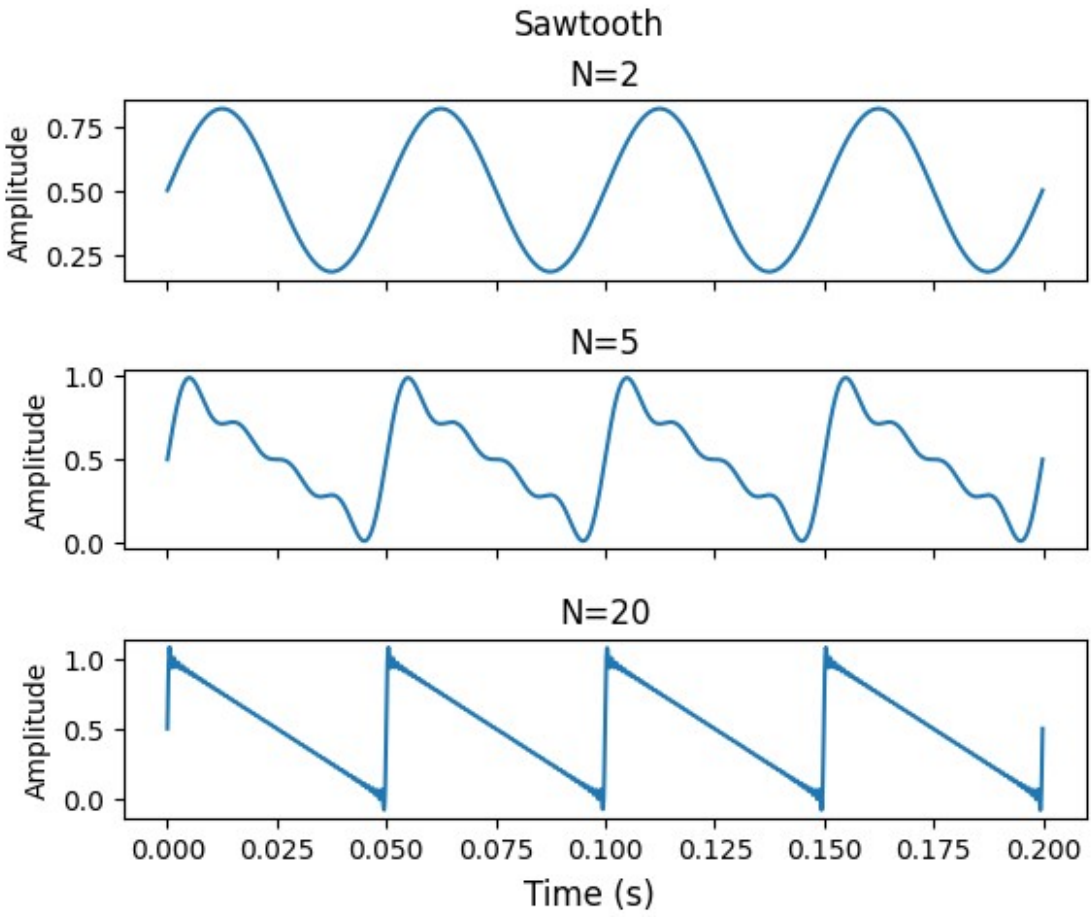
# Plot the 3 approximations
fig, (ax1, ax2, ax3) = plt.subplots(3,1, sharex=True)
fig.suptitle("Triangle")
fig.supxlabel("Time (s)")
fig.subplots_adjust(hspace=0.5)

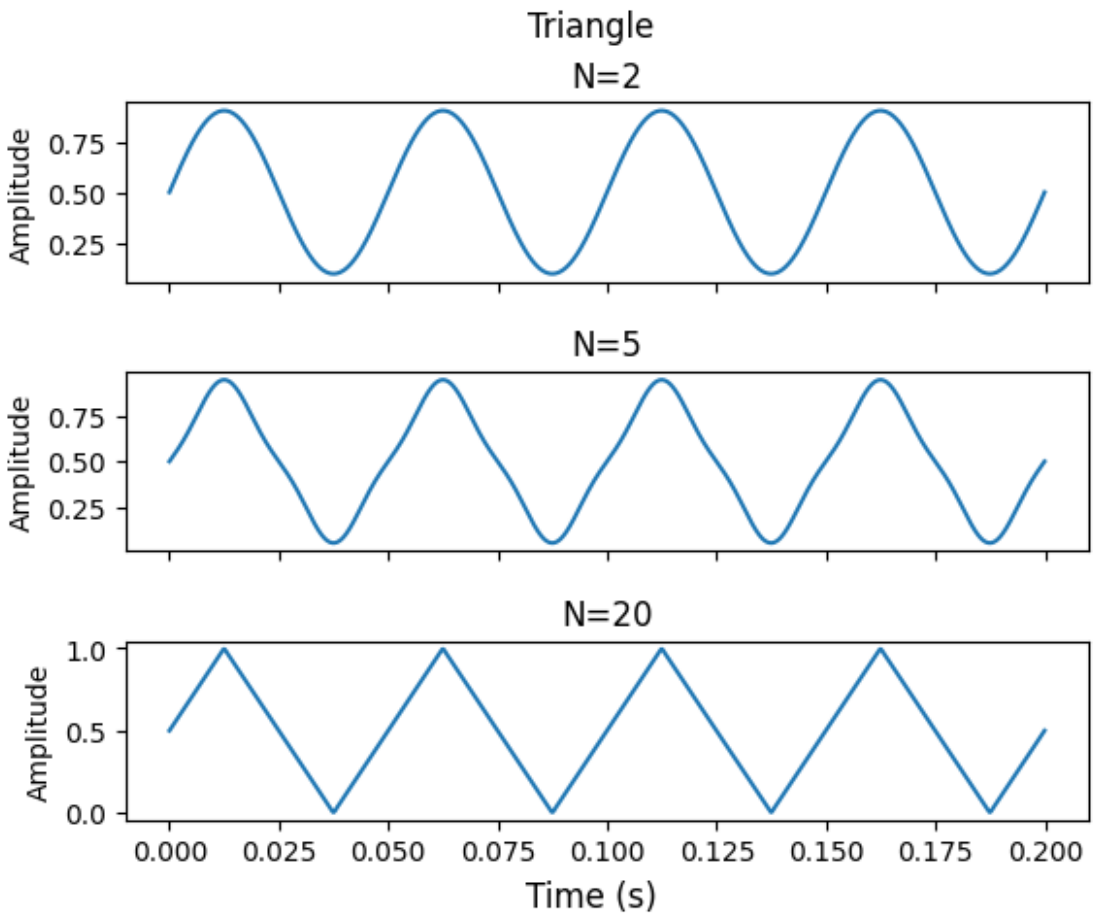
ax1.plot(t_vector, x1)
ax1.set_title("N=2")
ax1.set_ylabel("Amplitude")

ax2.plot(t_vector, x2)
ax2.set_title("N=5")
ax2.set_ylabel("Amplitude")

ax3.plot(t_vector, x3)
ax3.set_title("N=20")
ax3.set_ylabel("Amplitude")

Text(0, 0.5, 'Amplitude')
```


Sawtooth
N=2
Amplitude
0.75
0.50
0.25
N=5
Amplitude
1.0
0.5
0.0
N=20
Amplitude
1.0
0.5
0.0
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
Time (s)



Discussion

You should have noticed that the second signal converges more quickly. Discuss the two reasons for this.

One reason the triangle wave converges faster is that its continuous, the discontinuities in the sawtooth wave make it harder to be represented by the fourier series coeffecients. Another reason for the triangle wave converging faster is that looking at the Fourier coefficients from the formula above, the magnitude of the coefficients for the triangular wave decrease at a much faster rate as k increases compared to the sawtooth coefficients. This means that the triangular signal depents a lot less on higher frequency terms/harmonics, and therefore converges faster (with fewer frequency terms) than the sawtooth.

Assignment 2 -- Synthesizing a musical note

In this assignment, you will use the same synthesis equations to try to approximate a single note from a horn, which has the frequency characteristics illustrated below. Download the file horn11short.wav from the google drive to compare your synthesized version to the original.

Figure below shows the frequency component of a note played by a horn. image.png

A. Read in the horn signal, and use the sampling rate f_s that you read in to create a time vector of length 100ms. Define the fundamental frequency to be f_0 = 335Hz. Create a signal that is a sinusoid at that frequency, and save it as a wav file.

B. Create a vector (or two) to characterize a_k using:

$$|a_k| : [2688, 1900, 316, 178, 78, 38]$$

$$\angle\, a_k : [-1.73, -1.45, 2.36, 2.30, -2.30, 1.13]$$

assuming $a_0 = 0$ and the first element of the vectors correspond to a_1. Use the function you created in part 1 to synthesize a signal, with f_s and f_0 above, and save it as a wav file. Because the phase and magnitude are now hard coded, you may need to modify your function from above to apply here, so it is recommended that you copy and rename the function into another cell to make your debugging easier.

C. Plot the 100ms section of the original file starting at 200ms with a plot of the synthesized signal in a 2×1 plot.

D. Play the original file, the single tone, and the 6-tone approximation in series.

```
# Input: t = time in seconds, fs = sampling rate, a_mag = magnitude of
a_k, a_phase = phase of a_k, w = fundamental frequency
def fourier_series_2(t, fs, a_mag, a_phase, w):
    samples = int(t*fs)
    # avg value array with number of samples as t*f
    x_t = np.ones(samples) * a_mag[0]
    # time vector
    t = np.linspace(0, t, samples)
    # build fourier series
    for i in range(1, len(a_mag)):
        x_t += (2*a_mag[i]*np.cos(i*w*t + a_phase[i]))
    # return x(t)
    return x_t

# Assignment 2 - Synthesizing a musical note

# Part A - Reading the signal and finding parameters
# Read the horn signal and find fs
# Define f0 (2piw) as 335 Hz
# Create a signal that is just a sinewave at f0 over 100ms using the
function created in Assignment 1
    #fs1, x1t = wav.read("train32.wav")
from google.colab import drive
drive.mount('/content/drive/')
fs, horn = wavfile.read("/content/drive/MyDrive/horn11short.wav")
# print(fs)

f0 = 335
```

```
t = 0.1
t_vector = np.linspace(0, t, int(t*fs))
sine_wave = np.sin(2*np.pi*f0*t_vector)
sine_write = (sine_wave / max(abs(sine_wave)) *
32767).astype(np.int16)
wavfile.write("sine.wav", fs, sine_write)

# Part B - Initialize the parameters for your artificial curve.
# Create the "a" vector using the parameters given above
# Use the function created in Assignment 1 to create the signal.
a_mag = [0, 2688, 1900, 316, 178, 78, 38]
a_phase = [0, -1.73,-1.45,2.36,2.30,-2.30,1.13]
t2 = 0.3
t_vector2 = np.linspace(0, t2, int(t2*fs))
fourier_series_2b = fourier_series_2(t2, fs, a_mag, a_phase,
2*np.pi*f0)
fourier_write = (fourier_series_2b / max(abs(fourier_series_2b)) *
32767).astype(np.int16)
wavfile.write("fourier.wav", fs, fourier_write)

# Part C - Compare the artificial signal with original signal
# Find the part of the original signal between 200ms and 300ms
# Plot this along with your artificial signal
t_horn = len(horn) / fs
t_vector3 = np.linspace(0, t_horn, int(t_horn*fs))

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
fig.supxlabel("Time (s)")

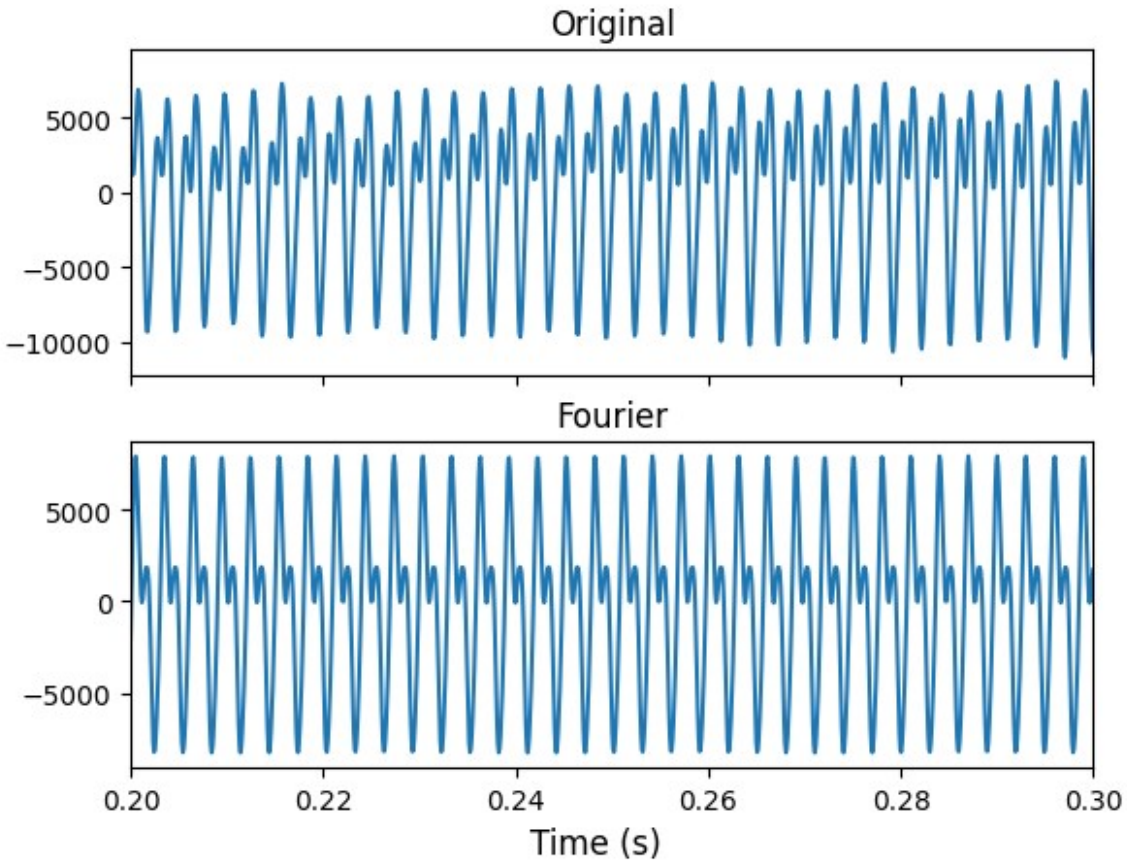

ax1.plot(t_vector3, horn)
ax1.set_xlim(0.2, 0.3)
ax1.set_title("Original")

ax2.plot(t_vector2, fourier_series_2b)
ax2.set_xlim(0.2, 0.3)
ax2.set_title("Fourier")


# Part D - Make some noise
# Play the original part, sinewave and 6-tone approximations one by
one in front of the TA
display(Audio("sine.wav"))
display(Audio("/content/drive/MyDrive/horn11short.wav"))
display(Audio("fourier.wav"))
```

```
Drive already mounted at /content/drive/; to attempt to forcibly
remount, call drive.mount("/content/drive/", force_remount=True).
11025
```

```
<IPython.lib.display.Audio object>
<IPython.lib.display.Audio object>
<IPython.lib.display.Audio object>
```



Discussion

The approximation does not sound quite like the original signal and the plot should look pretty different. The difference in sound is in part due to multiple factors, including the truncated approximation, imperfect estimate of the parameters, and the fact that the original signal is not perfectly periodic. Try adjusting some parameters and determine what you think is the main source of distortion.

(ASSUMING f0 IS FIXED) There are several reasons for the distortions, from truncated Fouier seires, imperfect parameters, and from the approximation not accounting for the envolope of the amplitude signal of the horn. The main source of distortion when sampling is most likely the truncated approximation. While changing the values of the the magnitude and phase values to be much more rounded didn't affect the sound noticably, when we reduced the number of harmonics in the Fourier series, the sound became discernably flatter (the timbre/complexity of the sound went away).